Introduction to MPI and MPI.jl

Juan Ignacio Polanco

ANF Julia4HPC: Formation au Langage Julia pour le Calcul Haute Performance

Fréjus, 16 October 2026

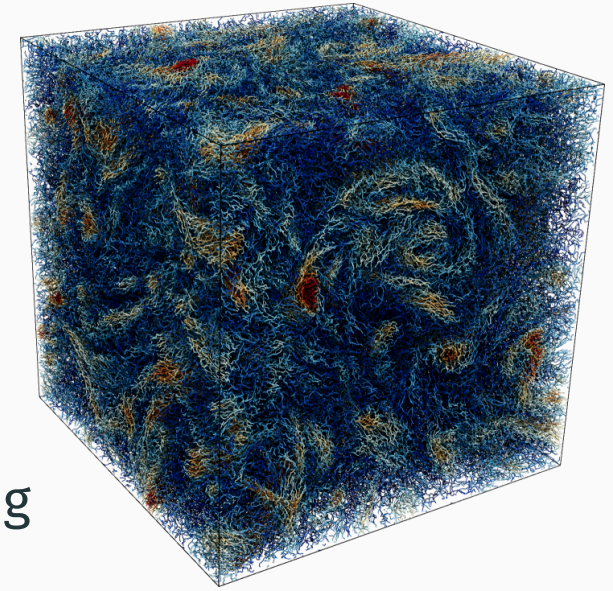








About myself



Researcher in fluid dynamics (LEGI, Grenoble)

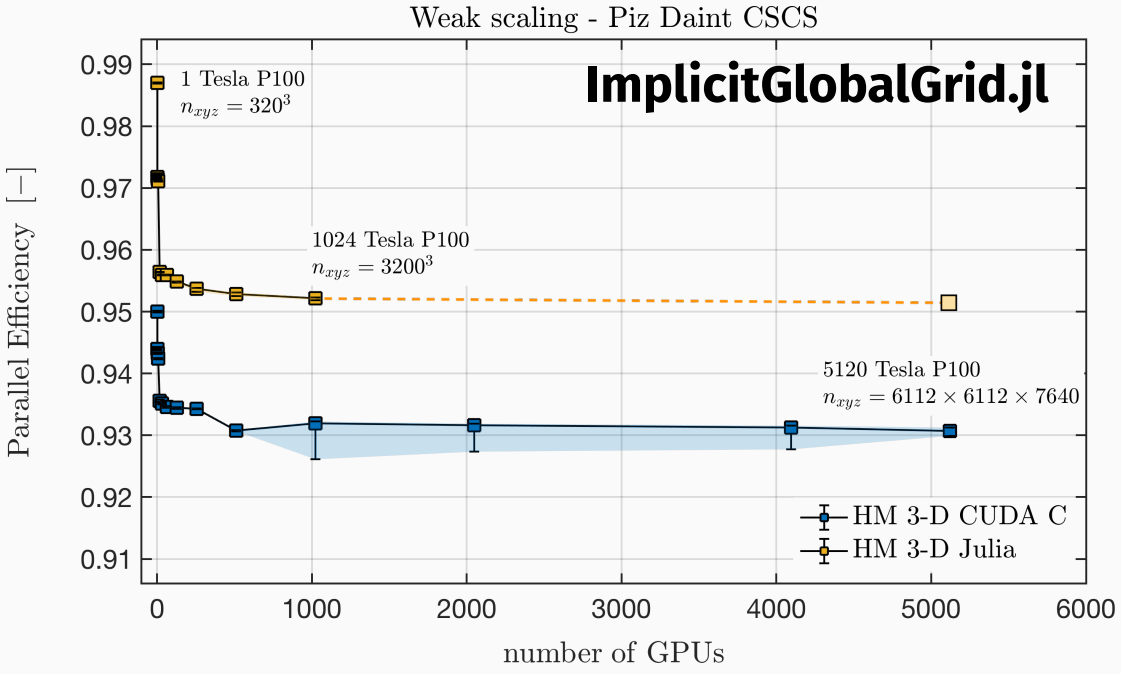
Long-time Julia user (since 2013 or 2014, Julia v0.3?)

Developer of open-source Julia packages for scientific computing

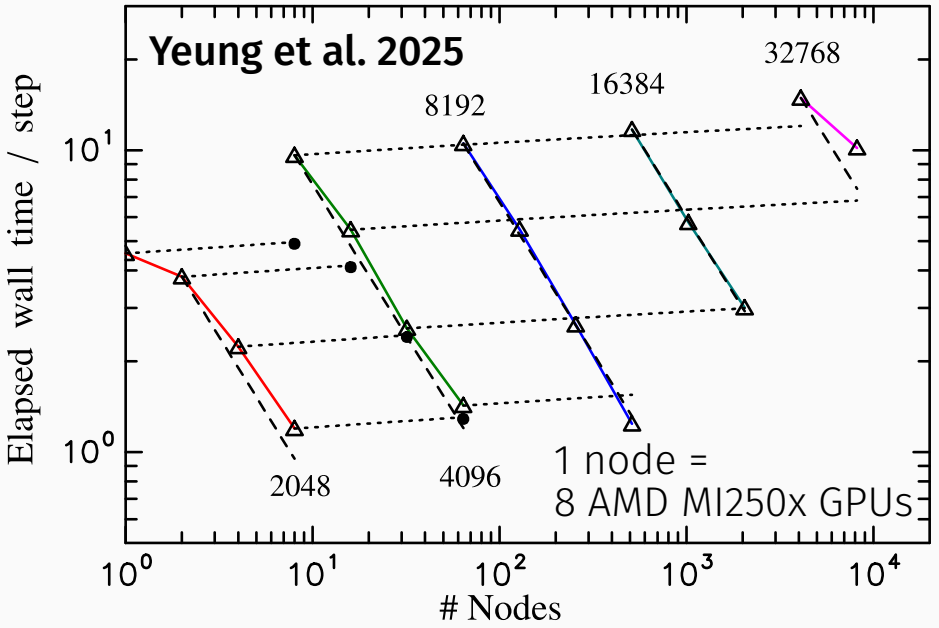
- **WriteVTK.jl** — scientific visualisation
- **PencilArrays.jl** / **PencilFFTs.jl** — distributed structured arrays with MPI
- **BSplineKit.jl** — spline interpolation and approximation
- **NonuniformFFTs.jl** — FFTs on nonuniform points (CPU and GPU)
- **VortexPasta.jl** — vortex filament solver (CPU and GPU)

What is MPI?

- It's the **de facto standard** for large-scale parallel computing
- Allows to scale over several thousands of CPU cores and GPUs
- **Distributed-memory parallelism:** each process has its own memory

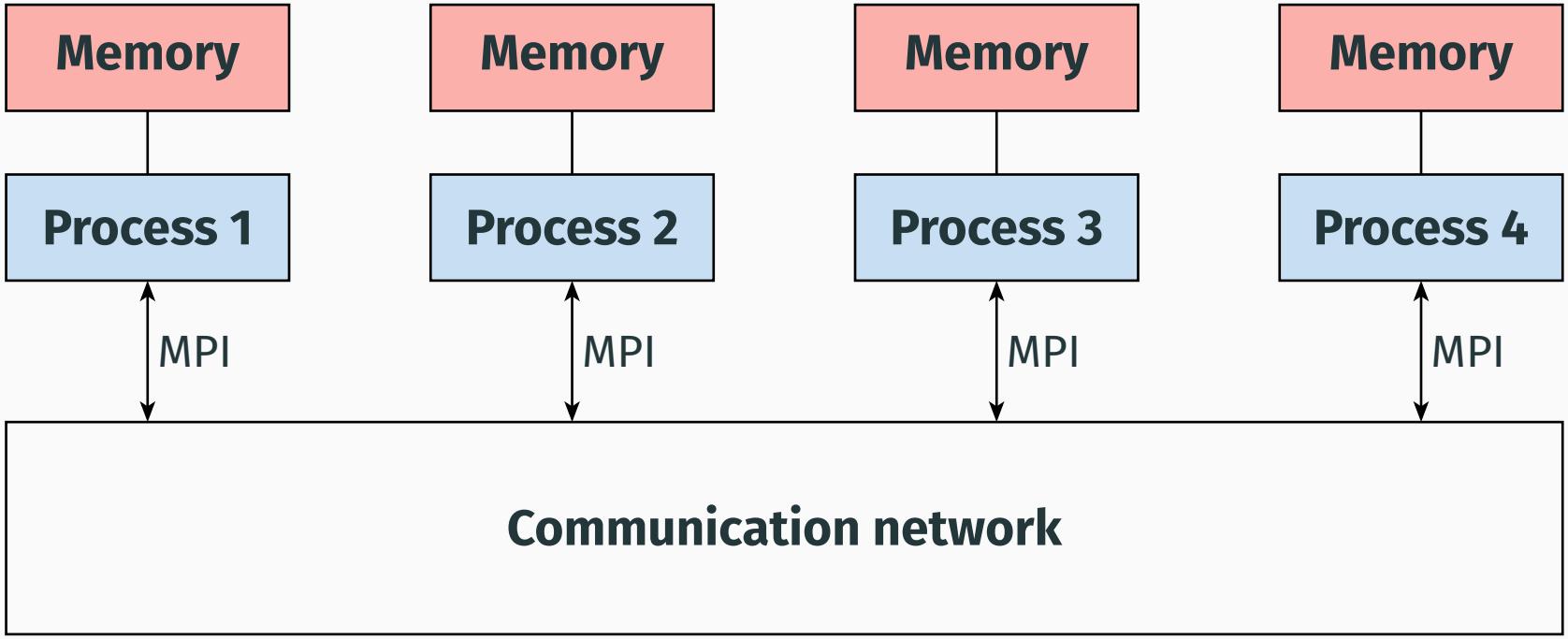


Turbulence simulations on Frontier
Up to $32{,}768^3$ grid points
Fortran + MPI + OpenMP offloading



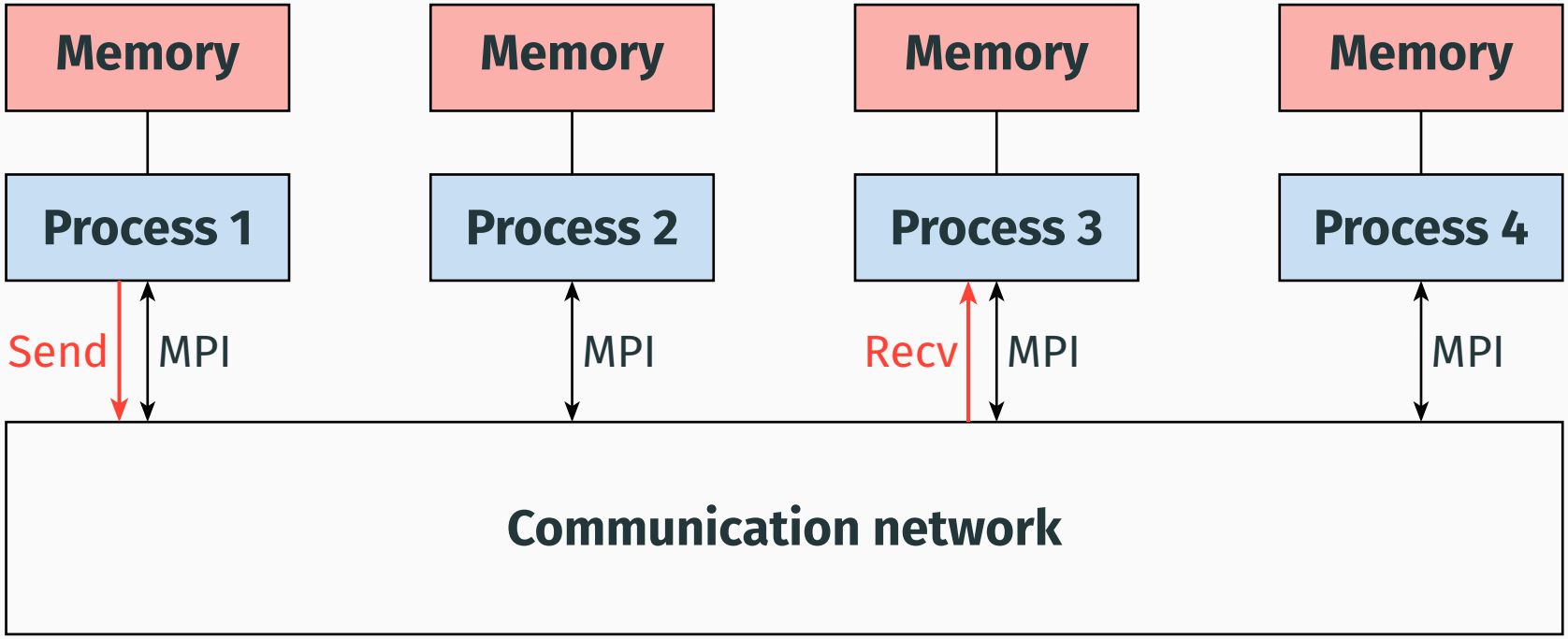
The MPI programming model

Single program, multiple data (SPMD): each process runs the same program on different data



The MPI programming model

Single program, multiple data (SPMD): each process runs the same program on different data



MPI (and MPI.jl) basics

Basic MPI.jl example

hello_world.jl

```
using MPI

MPI.Init()

comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

println(
  "Hello world, I am $rank of $nranks"
)

MPI.Finalize()
```

Running the script

```
# Add MPI.jl to the local project
$ julia --project=. -e '
    using Pkg
    Pkg.add("MPI")
  '


# Run script on 4 MPI processes
$ mpirun -n 4 \
      julia --project=. hello_world.jl

# Output
> Hello world, I am 1 of 4
> Hello world, I am 0 of 4
> Hello world, I am 3 of 4
> Hello world, I am 2 of 4
```

Basic point-to-point communication

point_to_point.jl

```
using MPI
MPI.Init()
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

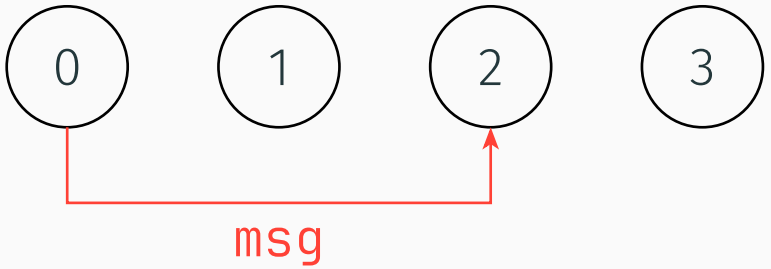
msg = [rank, rank]

if rank == 0
    MPI.Send(msg, comm; dest = 2)
elseif rank == 2
    MPI.Recv!(msg, comm; source = 0)
end
@show rank, msg

MPI.Finalize()
```

Output

```
> (rank, msg) = (1, [1, 1])
> (rank, msg) = (3, [3, 3])
> (rank, msg) = (0, [0, 0])
> (rank, msg) = (2, [0, 0])
```



A naïve example of cyclic communication (might deadlock)

cyclic_deadlock.jl

```
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

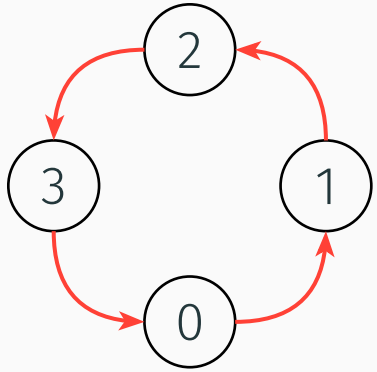
msg = [rank, rank]
right = mod(rank + 1, nranks)
left  = mod(rank - 1, nranks)

# These calls are BLOCKING
MPI.Send(msg, comm; dest = right)
MPI.Recv!(msg, comm; source = left)

@show rank, msg
```

Output (if it finishes)

```
> (rank, msg) = (1, [1, 1])
> (rank, msg) = (3, [3, 3])
> (rank, msg) = (0, [0, 0])
> (rank, msg) = (2, [0, 0])
```



A naïve example of cyclic communication (might deadlock)

cyclic_deadlock.jl

```
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

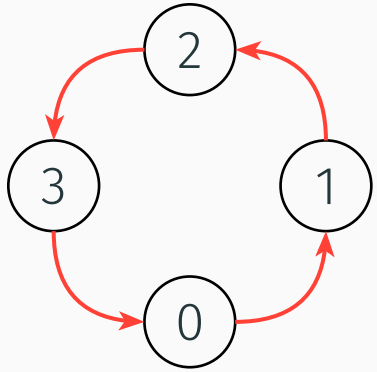
msg = [rank, rank]
right = mod(rank + 1, nranks)
left  = mod(rank - 1, nranks)

# These calls are BLOCKING
MPI.Send(msg, comm; dest = right)
MPI.Recv!(msg, comm; source = left)

@show rank, msg
```

Output (if it finishes)

```
> (rank, msg) = (1, [1, 1])
> (rank, msg) = (3, [3, 3])
> (rank, msg) = (0, [0, 0])
> (rank, msg) = (2, [0, 0])
```



In fact this might actually work if MPI decides to buffer the message in `Send` (MPI 4.1 standard, Sec. 3.4 Communication Modes)

A safer way of doing cyclic communications

cyclic_sendrecv.jl

```
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

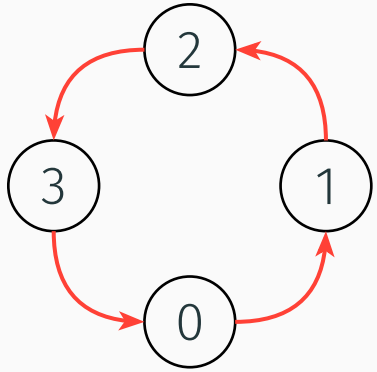
msg = [rank, rank]
right = mod(rank + 1, nranks)
left  = mod(rank - 1, nranks)

sendbuf = msg
recvbuf = similar(msg)
MPI.Sendrecv!(
  sendbuf, recvbuf, comm;
  dest = right, source = left,
)

@show rank, recvbuf
```

Output

```
> (rank, recvbuf) = (0, [3, 3])
> (rank, recvbuf) = (2, [1, 1])
> (rank, recvbuf) = (3, [2, 2])
> (rank, recvbuf) = (1, [0, 0])
```



A more flexible solution: use nonblocking operations

cyclic_nonblocking.jl

```
sendbuf = [rank, rank]
recvbuf = similar(sendbuf)
right = mod(rank + 1, nranks)
left  = mod(rank - 1, nranks)

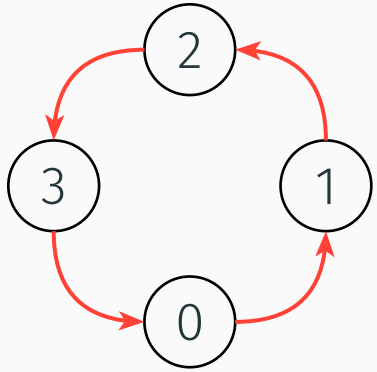
req_send = MPI.Request()
req_recv = MPI.Request()

MPI.Isend(sendbuf, comm, req_send;
          dest = right)
MPI.Irecv!(recvbuf, comm, req_recv;
           source = left)

# Wait for all operations to complete
MPI.Waitall([req_send, req_recv])
@show rank, recvbuf
```

Output

```
> (rank, recvbuf) = (1, [1, 1])
> (rank, recvbuf) = (3, [3, 3])
> (rank, recvbuf) = (0, [0, 0])
> (rank, recvbuf) = (2, [0, 0])
```



Collective communications

MPI broadcast

One MPI rank sends data to all other ranks.

bcast.jl

```
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

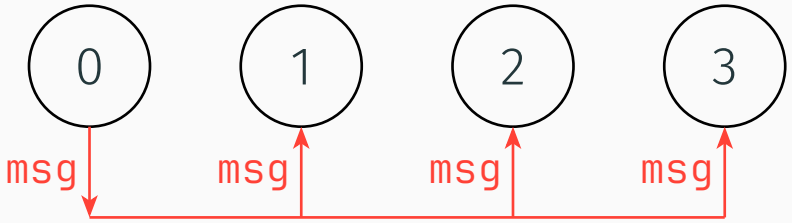
msg = [rank, rank]

# Send from rank 0 to all other ranks
MPI.Bcast!(msg, comm; root = 0)

# Now all ranks have [0, 0]
@show rank, msg
```

Output

```
> (rank, msg) = (0, [0, 0])
> (rank, msg) = (1, [0, 0])
> (rank, msg) = (3, [0, 0])
> (rank, msg) = (2, [0, 0])
```



MPI reduction

Perform binary operation across all MPI ranks (like `Base.reduce`).

reduce.jl

```
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

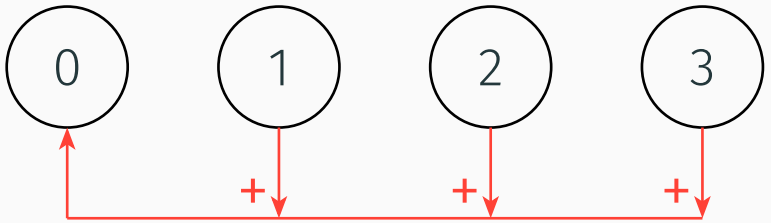
data = [rank, rank]

# Sum values from all ranks
MPI.Reduce!(data, +, comm; root = 0)

# Now rank 0 has the result
@show rank, data
```

Output

```
> (rank, data) = (0, [6, 6])
> (rank, data) = (1, [1, 1])
> (rank, data) = (2, [2, 2])
> (rank, data) = (3, [3, 3])
```



See also: `MPI.Allreduce!`, `MPI.Scan!` (like `Base.accumulate`)

MPI I/O

Writing MPI distributed data to a single binary file

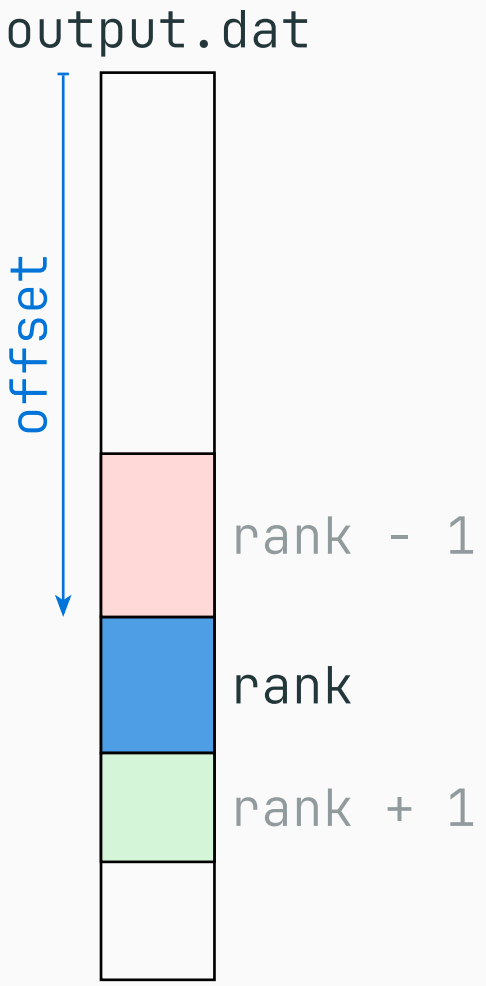
mpi_io.jl

```
rank = MPI.Comm_rank(comm)
data = [rank, rank]  # to be written by this process
T = eltype(data)

# Open file for writing
io = MPI.File.open(comm, "output.dat"; write = true)

# Write at an offset ("all" = collective)
offset = sizeof(T) * length(data) * rank
MPI.File.write_at_all(io, offset, data)

close(io)
```



One can also write to non contiguous blocks using custom data type + `MPI.File.set_view!` + `MPI.File.write_all` (advanced!)

Reading the data back

mpi_io.jl

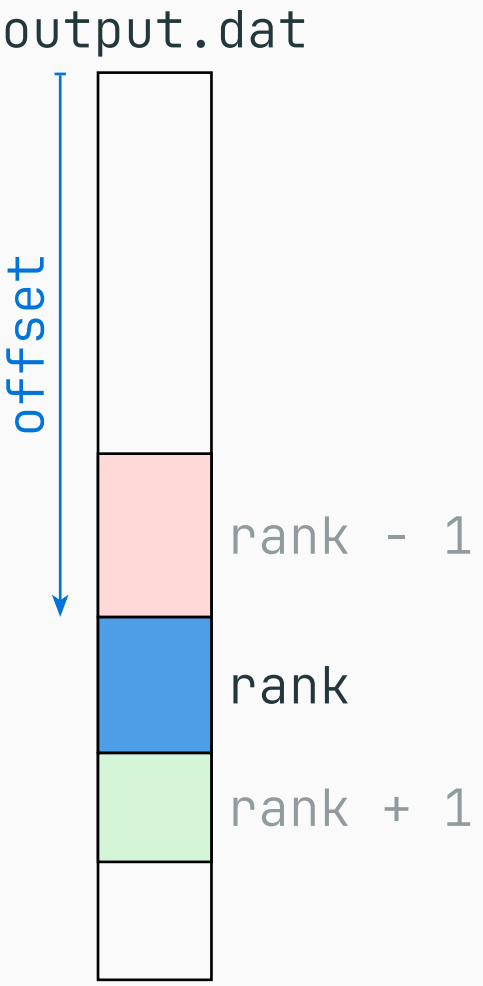
```
rank = MPI.Comm_rank(comm)
T = Int
data = Array{T}(undef, 2)  # to be read

# Open file for reading
io = MPI.File.open(comm, "output.dat"; read = true)

# Read at an offset ("all" = collective)
offset = sizeof(T) * length(data) * rank
MPI.File.read_at_all!(io, offset, data)

close(io)
```

Alternatively, one can use **parallel HDF5** via HDF5.jl + MPI.jl
(but it can be tricky to set-up)



The Julia MPI ecosystem

The Julia MPI ecosystem – A few relevant packages

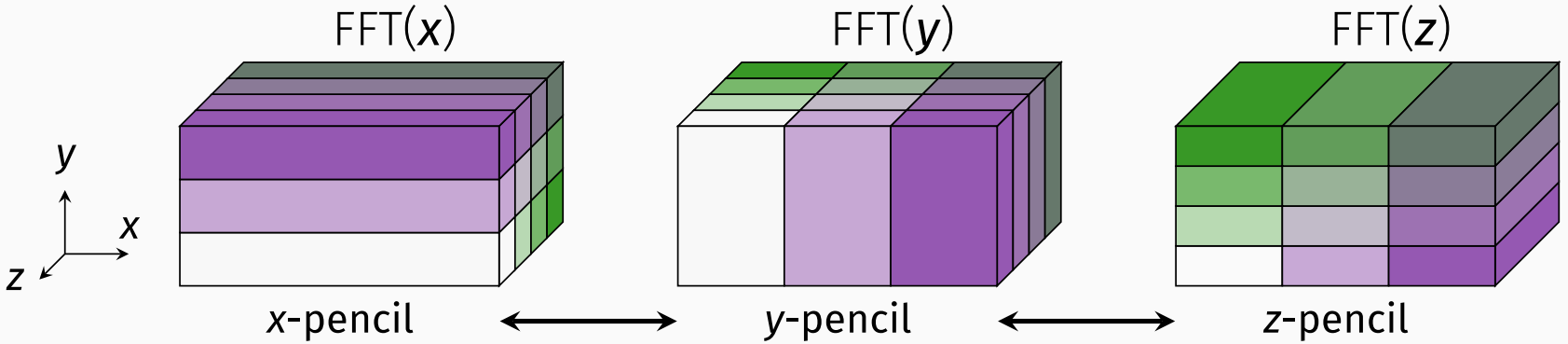
Libraries

ImplicitGlobalGrid.jl / ParallelStencil.jl	Local operations, staggered grids
Chmy.jl	Finite differences, staggered grids
PencilArrays.jl / PencilFFTs.jl	Spectral methods, global operations, FFTs
HaloArrays.jl, MPIHaloArrays.jl	Domain decomposition, halo exchanges
PartitionedArrays.jl	Linear algebra, unstructured data
Elemental.jl	Dense and sparse linear algebra
PETSc.jl	Linear and nonlinear solvers, timesteppers, ...

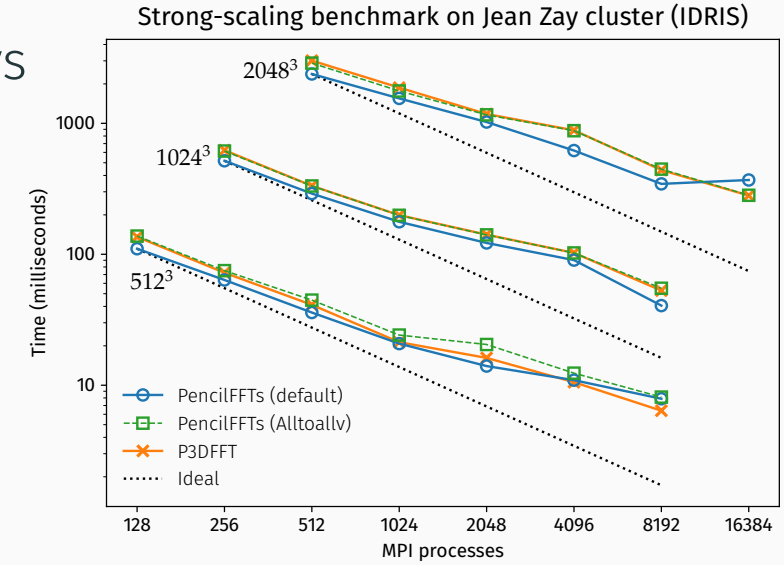
Some applications and solvers

ClimaCore.jl / Oceananigans.jl	Climate models, geophysical flows
Trixi.jl	Discontinuous Galerkin methods
GridapDistributed.jl	Finite element methods
Tarang.jl	Spectral methods, fluid dynamics

The PencilArrays.jl package



- Multidimensional decomposition of structured arrays (in arbitrary dimensions!)
- Transpositions between different decompositions
- Parallel I/O (raw MPI-IO and Parallel HDF5)
- Parallel FFTs via PencilFFTs.jl



Setting up Julia and MPI on Arctic

Loading Julia and MPI

For more details: `https://github.com/GroupeCalcul/JuliaFrejus2026`

~/.bashrc

```
export SHARE=/home/2500001/PROJETS/M26182/PARTAGE/
export MODULEPATH=$SHARE/privatemodules:$MODULEPATH
module load julia
module load cpe_env/gcc13.2.1-mpich8.1.30-milan-02.25
```

~/.bash_profile

```
if [ -f ~/.bashrc ]; then
  . ~/.bashrc
fi
```

Then:

```
$ julia --version
julia version 1.13.1

$ mpichversion
MPICH Version:      3.4a2
[...]
```

Using system MPI in Julia

Create local environment

```
$ mkdir mpi_tutorials
$ cd mpi_tutorials
$ julia --project=.
```

Set-up MPI in Julia

```
(mpi_tutorials) pkg> add MPI, MPIPreferences  # in pkg mode (tap ']' first)
julia> using MPIPreferences
julia> MPIPreferences.use_system_binary()
┌ Info: MPI implementation identified
│   libmpi = "libmpi_cray"
│   version_string = "MPI VERSION    : CRAY MPICH version 8.1.30.1 [...]"
└   impl = "CrayMPICH"
    [...]
```

→ Check for new `LocalPreferences.toml` file

Running an MPI job

hello_mpi.jl

```
using MPI
MPI.Init()
comm = MPI.COMM_WORLD
println("Hello world, I am $(MPI.Comm_rank(comm)) of $(MPI.Comm_size(comm))")
MPI.Barrier(comm)
MPI.Finalize()
```

```
$ srun --ntasks 4 --time 10:00 --partition ar_fin julia --project hello_mpi.jl
srun: job 3275679 queued and waiting for resources
srun: job 3275679 has been allocated resources
Hello world, I am 1 of 4
Hello world, I am 0 of 4
Hello world, I am 3 of 4
Hello world, I am 2 of 4
```

A basic SLURM script

launch.slurm → sbatch launch.slurm

```
#!/bin/bash
#SBATCH --job-name "mpi_example"
#SBATCH --output %x.%j.out
#SBATCH --error %x.%j.err
#SBATCH --partition ar_fin
#SBATCH --time 00:10:00
#SBATCH --ntasks 8             # number of MPI tasks
#SBATCH --cpus-per-task 1      # 1 CPU = 1 MPI task
#SBATCH --hint nomultithread   # disable hyperthreading

export SHARE=/home/2500001/PROJETS/M26182/PARTAGE/
export MODULEPATH=$SHARE/privatemodules:$MODULEPATH
module load cpe_env/gcc13.2.1-mpich8.1.30-milan-02.25
module load julia
module list
srun julia --project hello_mpi.jl
```

Exercises

1. Numerical approximation of definite integral

Estimate $4 \int_0^1 \sqrt{1 - x^2}\, \mathrm{d}x \; (= \pi)$ using the trapezoidal rule on $N + 1$ equispaced points.

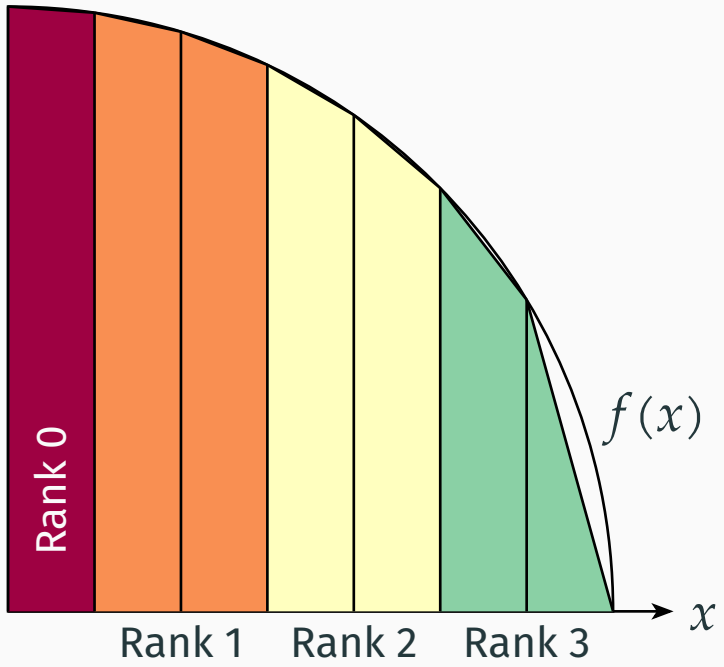
Idea: each process integrates over a subset of $[0, 1]$.

```
using MPI
comm = MPI.COMM_WORLD
nranks = MPI.Comm_size(comm)
rank = MPI.Comm_rank(comm)
N = 211  # prime number (to make things fun)
xs = range(0, 1; length = N + 1)  # evaluation points

# Determine local integration interval (avoid overlaps!)
i_begin = ...
i_end = ...

# Compute local integral
integ_local = sum(i_begin:i_end) do i
    xa, xb = xs[i], xs[i + 1]
    (f(xa) + f(xb)) * (xb - xa) / 2
end

# TODO: obtain global result
```



Inspiration taken from Carsten Bauer's lecture notes.

2. Monte Carlo estimation of π

Estimate π by randomly sampling the $[0,1]^2$ square

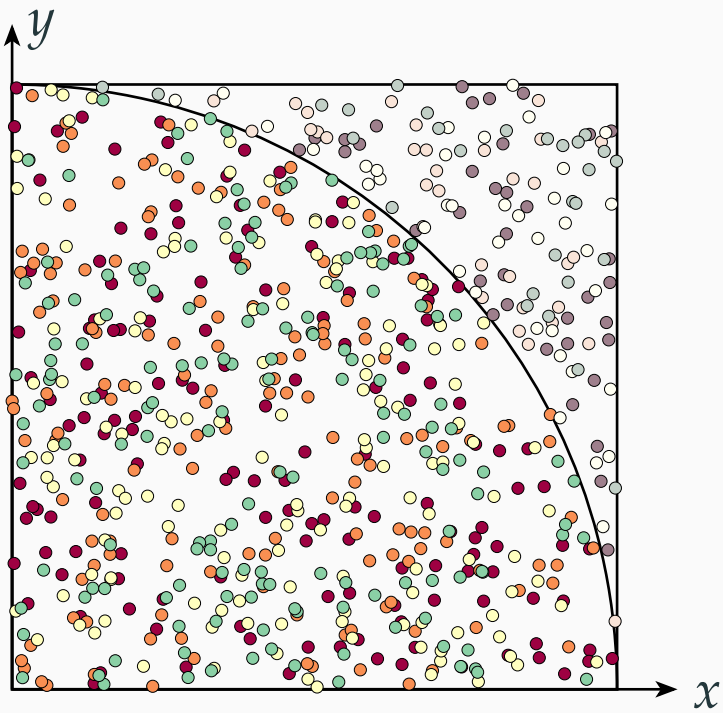
```julia
using MPI, Random
comm = MPI.COMM_WORLD
nranks = MPI.Comm_size(comm)
rank = MPI.Comm_rank(comm)
N_global = 1_000_000  # global number of evaluations

# Determine local number of evaluations
N_local = ...

# Make sure all ranks generate different numbers
rng = Random.default_rng()
Random.seed!(rng, 42 + rank)

# Compute local estimate
sum_local = sum(1:N_local) do n
    x = rand(rng, Float64)
    y = rand(rng, Float64)
    float(x^2 + y^2 < 1)  # = 1 if within the circle
end  # result should be close to (π/4) * N_local

# TODO: obtain global result
```



2. Monte Carlo estimation of π

Estimate π by randomly sampling the $[0,1]^2$ square

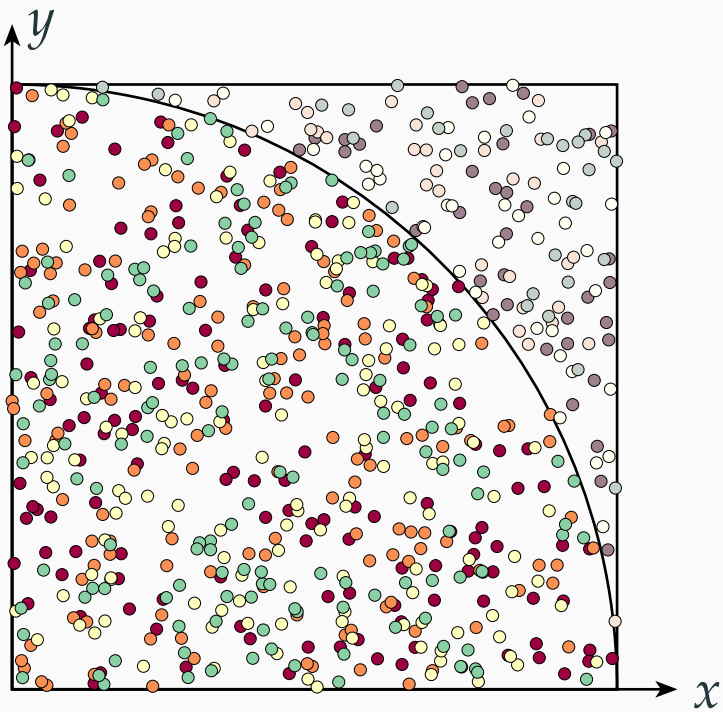
```
using MPI, Random
comm = MPI.COMM_WORLD
nranks = MPI.Comm_size(comm)
rank = MPI.Comm_rank(comm)
N_global = 1_000_000  # global number of evaluations

# Determine local number of evaluations
N_local = ...

# Make sure all ranks generate different numbers
rng = Random.default_rng()
Random.seed!(rng, 42 + rank)

# Compute local estimate
sum_local = sum(1:N_local) do n
    x = rand(rng, Float64)
    y = rand(rng, Float64)
    float(x^2 + y^2 < 1)  # = 1 if within the circle
end  # result should be close to (π/4) * N_local

# TODO: obtain global result
```



Problem: results depend on number of processes

Inspiration taken from Carsten Bauer's lecture notes.

3. “Reproducible” random number generation can be tricky

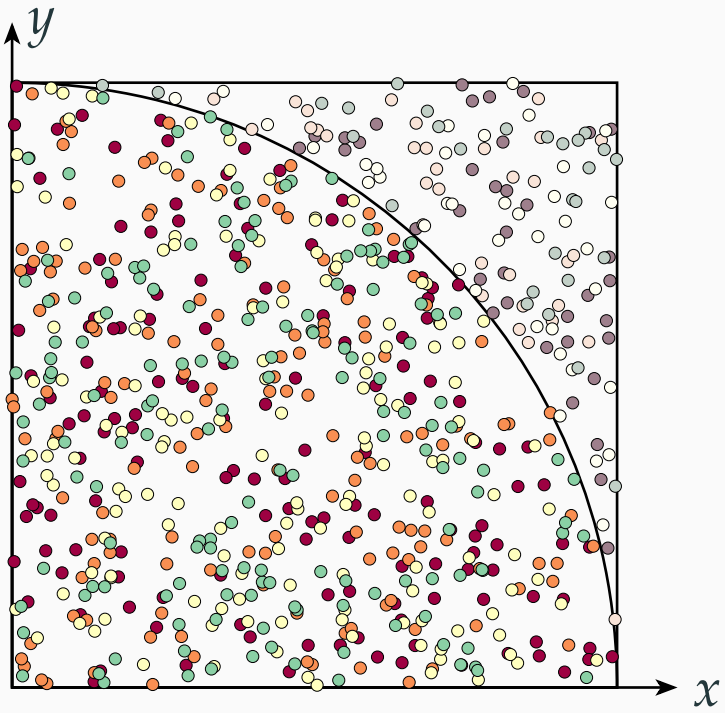
```julia
N_global = 1_000_000  # global number of evaluations
i_begin = N_global * rank ÷ nranks + 1
i_end = N_global * (rank + 1) ÷ nranks
N_local = i_end - i_begin + 1

# Make sure all ranks generate different numbers
rng = Random.default_rng()
Random.seed!(rng, 42)  # same for all ranks!

# Advance RNG to start of this rank (inefficient!)
for i in 1:(i_begin - 1)
    x = rand(rng, Float64)  # discarded
    y = rand(rng, Float64)  # discarded
end

sum_local = sum(1:N_local) do n
    x = rand(rng, Float64)
    y = rand(rng, Float64)
    float(x^2 + y^2 < 1)  # = 1 if within the circle
end  # result should be close to (π/4) * N_local

sum_global = MPI.Allreduce(sum_local, +, comm)
π_global = sum_global * 4 / N_global
```



3. “Reproducible” random number generation can be tricky

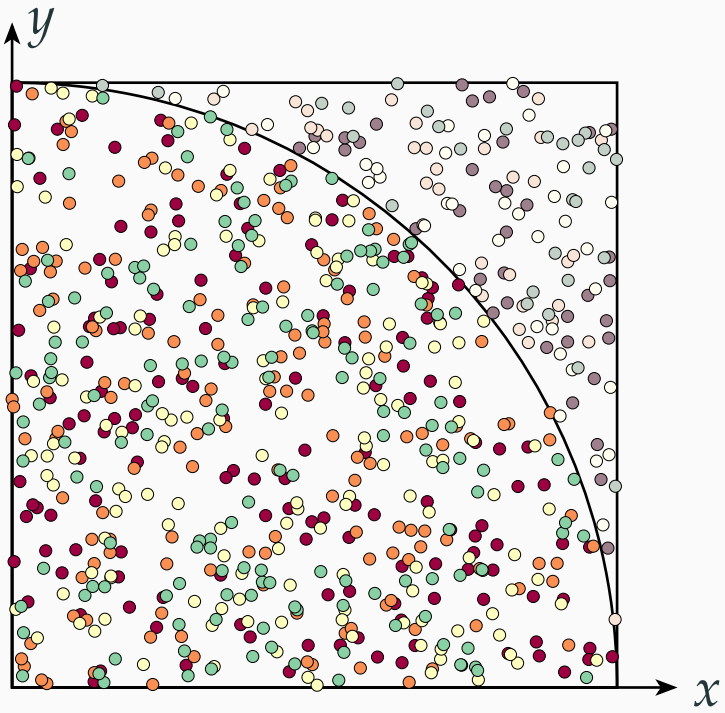
```julia
N_global = 1_000_000  # global number of evaluations
i_begin = N_global * rank ÷ nranks + 1
i_end = N_global * (rank + 1) ÷ nranks
N_local = i_end - i_begin + 1

# Make sure all ranks generate different numbers
rng = Random.default_rng()
Random.seed!(rng, 42)  # same for all ranks!

# Advance RNG to start of this rank (inefficient!)
for i in 1:(i_begin - 1)
    x = rand(rng, Float64)  # discarded
    y = rand(rng, Float64)  # discarded
end

sum_local = sum(1:N_local) do n
    x = rand(rng, Float64)
    y = rand(rng, Float64)
    float(x^2 + y^2 < 1)  # = 1 if within the circle
end  # result should be close to (π/4) * N_local

sum_global = MPI.Allreduce(sum_local, +, comm)
π_global = sum_global * 4 / N_global
```



Better solution: use RNG from RandomNumbers.jl

Inspiration taken from Carsten Bauer’s lecture notes.

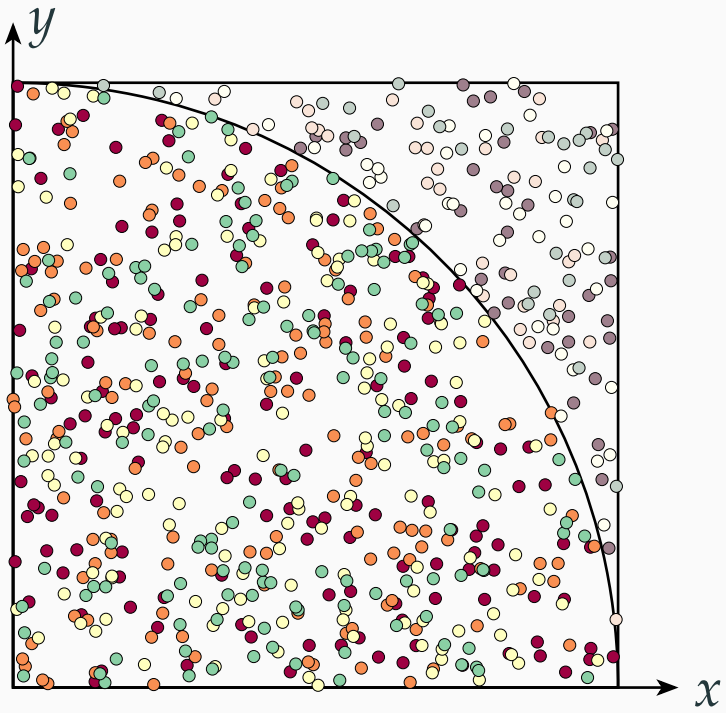
3′. Using RandomNumbers.jl + advance!

```julia
using MPI, Random
using RandomNumbers: PCG
N_global = 1_000_000  # global number of evaluations
i_begin = N_global * rank ÷ nranks + 1
i_end = N_global * (rank + 1) ÷ nranks
N_local = i_end - i_begin + 1

# Make results reproducible if one changes the number of
procs.
rng = PCG.PCGStateOneseq()  # from RandomNumbers
Random.seed!(rng, 42)  # same for all ranks!

# Advance RNG to start of this rank (fast!)
nrand_prev = 2 * (i_begin - 1)
PCG.advance!(rng, nrand_prev)

sum_local = sum(1:N_local) do n
    x = rand(rng, Float64)
    y = rand(rng, Float64)
    float(x^2 + y^2 < 1)  # = 1 if within the circle
end
# etc...
```



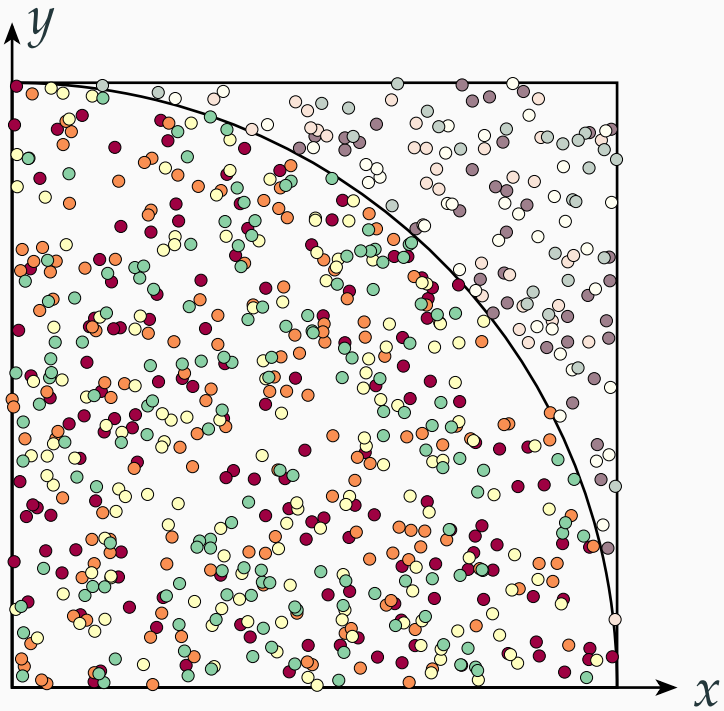
3′. Using RandomNumbers.jl + advance!

```julia
using MPI, Random
using RandomNumbers: PCG
N_global = 1_000_000  # global number of evaluations
i_begin = N_global * rank ÷ nranks + 1
i_end = N_global * (rank + 1) ÷ nranks
N_local = i_end - i_begin + 1

# Make results reproducible if one changes the number of
procs.
rng = PCG.PCGStateOneseq()  # from RandomNumbers
Random.seed!(rng, 42)  # same for all ranks!

# Advance RNG to start of this rank (fast!)
nrand_prev = 2 * (i_begin - 1)
PCG.advance!(rng, nrand_prev)

sum_local = sum(1:N_local) do n
    x = rand(rng, Float64)
    y = rand(rng, Float64)
    float(x^2 + y^2 < 1)  # = 1 if within the circle
end
# etc...
```



Better solution: use RNG from RandomNumbers.jl

Inspiration taken from Carsten Bauer's lecture notes.

4. Distributed sort + output to binary file

Objective: sort random values and write them to a binary file

Steps:
1. Rank 0 generates random numbers, then sends them to all ranks
2. Define "local" domain and perform local sort
3. Write to binary file (with the right offset!)

```
# 1. Generate random numbers
N_global = 1_000  # number of random values
vs = zeros(N_global)  # to be filled!
if rank == 0
    rand!(vs)
end
# TODO: send to other ranks

# 2. Perform local sort
v_begin = rank / nranks
v_end = (rank + 1) / nranks
vs_local = filter(vs) do v
    v_begin <= v < v_end
end
sort!(vs_local)
```

```
# 3. Write to binary file using MPI-IO
# TODO: determine the local offset (see MPI.Scan)
offset = ...

file = MPI.File.open(comm, "sorted.dat";
                     write = true)
MPI.File.write_at_all(file, offset, vs_local)
close(file)

# 4. Verify
if rank == 0
    values_sorted = similar(values)
    read!("sorted.dat", values_sorted)
    @show issorted(values_sorted)
end
```

5. Solving the 2D heat equation using FFTs

$$\partial_t u(\boldsymbol{x}, t) = \nu \nabla^2 u(\boldsymbol{x}, t) \quad \text{for} \quad \boldsymbol{x} \in [0, 2\pi]^2$$

Periodic solution: $u(\boldsymbol{x}, t) = \sum_{\boldsymbol{k}} \hat{u}(\boldsymbol{k}, t)\, e^{i\boldsymbol{k}\cdot\boldsymbol{x}}$

$$\Rightarrow \partial_t \hat{u}(\boldsymbol{k}, t) = -\nu k^2 \hat{u}(\boldsymbol{k}, t), \quad k = |\boldsymbol{k}|$$

Solution: $\hat{u}(\boldsymbol{k}, t) = e^{-\nu k^2 t}\, \hat{u}(\boldsymbol{k}, 0)$

5. Solving the 2D heat equation using FFTs

$$\partial_t u(\boldsymbol{x}, t) = \nu \nabla^2 u(\boldsymbol{x}, t) \quad \text{for} \quad \boldsymbol{x} \in [0, 2\pi]^2$$

Periodic solution: $u(\boldsymbol{x}, t) = \sum_{\boldsymbol{k}} \hat{u}(\boldsymbol{k}, t)\, e^{i\boldsymbol{k}\cdot\boldsymbol{x}}$

$$\Rightarrow \partial_t \hat{u}(\boldsymbol{k}, t) = -\nu k^2 \hat{u}(\boldsymbol{k}, t), \quad k = |\boldsymbol{k}|$$

Solution: $\hat{u}(\boldsymbol{k}, t) = e^{-\nu k^2 t}\, \hat{u}(\boldsymbol{k}, 0)$

Basic steps:

1. Set initial condition u_{ij} on grid $\left(x_i, y_j\right)$
2. Perform 2D FFT: $u_{ij} \xrightarrow{\text{FFT}} \hat{u}_{ij}$ on grid $\left(k_i^x, k_j^y\right)$
3. Element by element multiplication: $\hat{u}_{ij} \longleftarrow e^{-\nu k_{ij}^2 t}\, \hat{u}_{ij}$
4. Perform inverse 2D FFT: $\hat{u}_{ij} \xrightarrow{\text{IFFT}} u_{ij}$

5. Solving the 2D heat equation using FFTs

$$\partial_t u(\boldsymbol{x}, t) = \nu \nabla^2 u(\boldsymbol{x}, t) \quad \text{for} \quad \boldsymbol{x} \in [0, 2\pi]^2$$

Periodic solution: $u(\boldsymbol{x}, t) = \sum_{\boldsymbol{k}} \hat{u}(\boldsymbol{k}, t)\, e^{i\boldsymbol{k}\cdot\boldsymbol{x}}$

$$\Rightarrow \partial_t \hat{u}(\boldsymbol{k}, t) = -\nu k^2 \hat{u}(\boldsymbol{k}, t), \quad k = |\boldsymbol{k}|$$

Solution: $\hat{u}(\boldsymbol{k}, t) = e^{-\nu k^2 t}\, \hat{u}(\boldsymbol{k}, 0)$

Basic steps:

1. Set initial condition u_{ij} on grid $\left(x_i, y_j\right)$
2. Perform 2D FFT: $u_{ij} \xrightarrow{\text{FFT}} \hat{u}_{ij}$ on grid $\left(k_i^x, k_j^y\right)$
3. Element by element multiplication: $\hat{u}_{ij} \longleftarrow e^{-\nu k_{ij}^2 t}\, \hat{u}_{ij}$
4. Perform inverse 2D FFT: $\hat{u}_{ij} \xrightarrow{\text{IFFT}} u_{ij}$

Problem: the FFT is a **global** operation

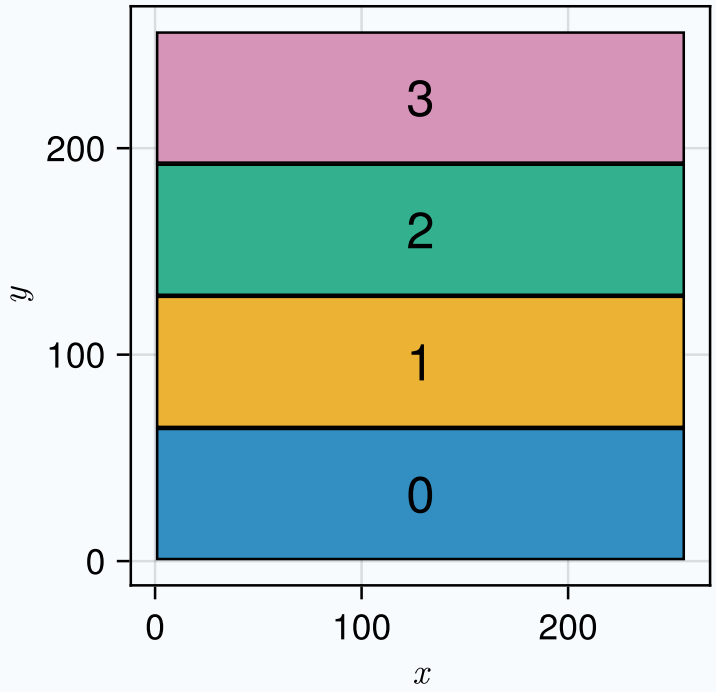
5. Solving the 2D heat equation using FFTs

1. Create domain decomposition

```julia
# Usual MPI stuff
using MPI
MPI.Init()
comm = MPI.COMM_WORLD
nranks = MPI.Comm_size(comm)
rank = MPI.Comm_rank(comm)

Ns = (256, 256)  # global grid size (Nx, Ny)

# Generate domain partition
# By default, the first dimension stays local
using PencilArrays
pen = Pencil(Ns, comm)  # = domain decomposition
visualise_decomposition("decomp_phys.svg", pen)
```



5. Solving the 2D heat equation using FFTs

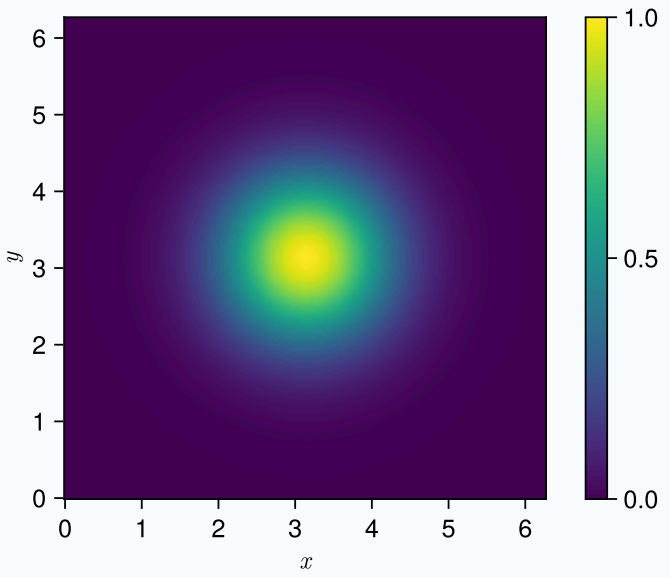
2. Define the geometry and set the initial condition

```julia
# Global domain
Ls = (2π, 2π)    # = Lx, Ly
xs_global = (
    range(0, Ls[1]; length = Ns[1] + 1)[1:(end - 1)],
    range(0, Ls[2]; length = Ns[2] + 1)[1:(end - 1)],
)

# Local domain
grid = localgrid(pen, xs_global)

# Initial condition (Gaussian)
u_init = PencilArray{Float64}(undef, pen)  # will contain local data
x₀, y₀, σ = Ls[1]/2, Ls[2]/2, Ls[1]/8
@. u_init = exp(-((grid.x - x₀)^2 + (grid.y - y₀)^2) / (2 * σ^2))

plot_field("u_initial.svg", xs_global, u_init)
```



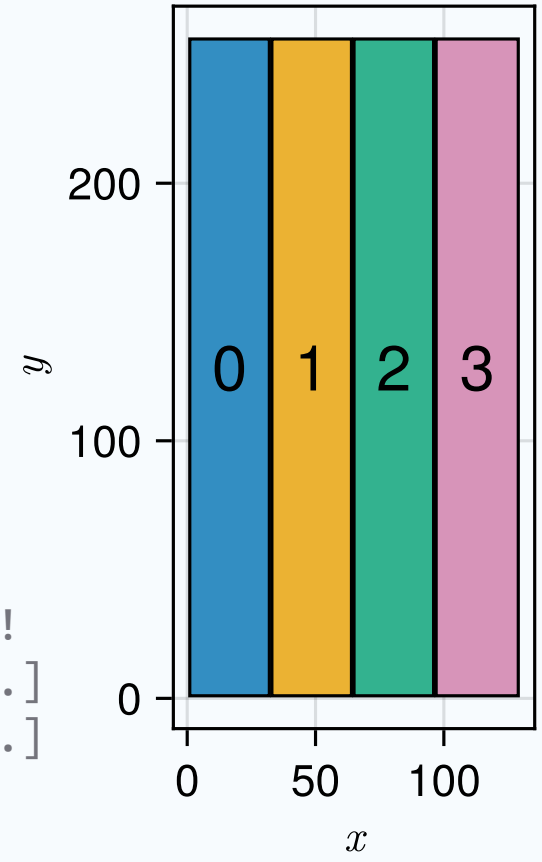
5. Solving the 2D heat equation using FFTs

3. Perform parallel FFT and define spectral domain

```julia
# Create real-to-complex FFT plan and apply it
using PencilFFTs
plan = PencilFFTPlan(u_init, Transforms.RFFT())
u_hat = plan * u_init

# Let's look at the decomposition of u_hat
pen_f = pencil(u_hat)  # decomposition in Fourier space
visualise_decomposition("decomp_fourier.svg", pen_f)

# Define grid in Fourier space (wavenumbers kx, ky)
ks_global = (
    # The first dimension uses `rfftfreq` instead of `fftfreq`!
    rfftfreq(Ns[1], 2π * Ns[1] / Ls[1]), # = [kx[1], kx[2], ...]
     fftfreq(Ns[2], 2π * Ns[2] / Ls[2]), # = [ky[1], ky[2], ...]
)
grid_f = localgrid(pen_f, ks_global)  # local grid (kx, ky)
```



5. Solving the 2D heat equation using FFTs

4. Apply diffusion operator and back to physical space

```
ν = 0.2  # diffusivity
t = 1.0  # final time

# We can do a simple `for` loop over the local data
for I in CartesianIndices(u_hat)
    i, j = Tuple(I)
    kx, ky = grid_f[I]
    k² = kx^2 + ky^2
    op = exp(-ν * k² * t)      # diffusion operator
    u_hat[I] = op * u_hat[I]  # advance û(0) → û(t)
end

# Transform back to physical space
u_final = plan \ u_hat

plot_field("u_final.svg", xs_global, u_final)
```

